

ViT-JAX Distributed Scaling

Data-parallel Vision Transformer training on TPU v6e-8

Scaling behaviour & straggler tax

Michael Wang

Team Tensor

Spring 2026

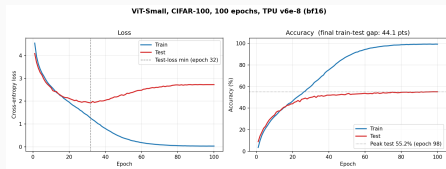
CS 390 — Distributed Systems · Duke University

What I built and why

A systems study of synchronous data parallelism, using ViT-Small (9.55 M params) on CIFAR-100 in JAX/Flax, running on TPU v6e-8 (8 chips, bf16).

Two experiments on the same testbed:

1. **Scaling** — how throughput grows with device count.



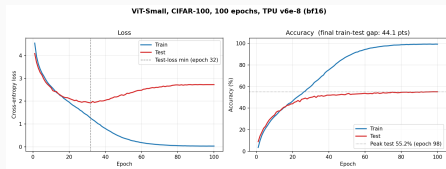
Train/test loss & accuracy, 100 epochs.

What I built and why

A systems study of synchronous data parallelism, using ViT-Small (9.55 M params) on CIFAR-100 in JAX/Flax, running on TPU v6e-8 (8 chips, bf16).

Two experiments on the same testbed:

1. **Scaling** — how throughput grows with device count.
2. **Stragglers** — what one slow chip costs the cluster.



Train/test loss & accuracy, 100 epochs.

What I built and why

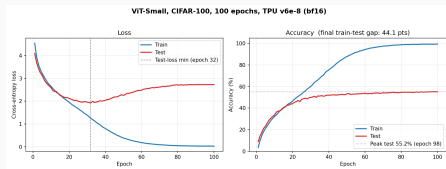
A systems study of synchronous data parallelism, using ViT-Small (9.55 M params) on CIFAR-100 in JAX/Flax, running on TPU v6e-8 (8 chips, bf16).

Two experiments on the same testbed:

1. **Scaling** — how throughput grows with device count.
2. **Stragglers** — what one slow chip costs the cluster.

Headline training run:

- 100 epochs in 421 s
- 55.04 % top-1, 80.18 % top-5
- $\approx$ 15 000 images/sec sustained



Train/test loss & accuracy, 100 epochs.

What I built and why

A systems study of synchronous data parallelism, using ViT-Small (9.55 M params) on CIFAR-100 in JAX/Flax, running on TPU v6e-8 (8 chips, bf16).

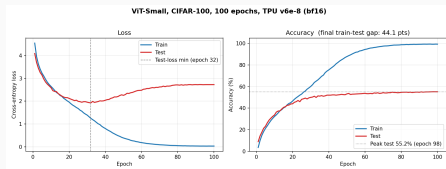
Two experiments on the same testbed:

1. **Scaling** — how throughput grows with device count.
2. **Stragglers** — what one slow chip costs the cluster.

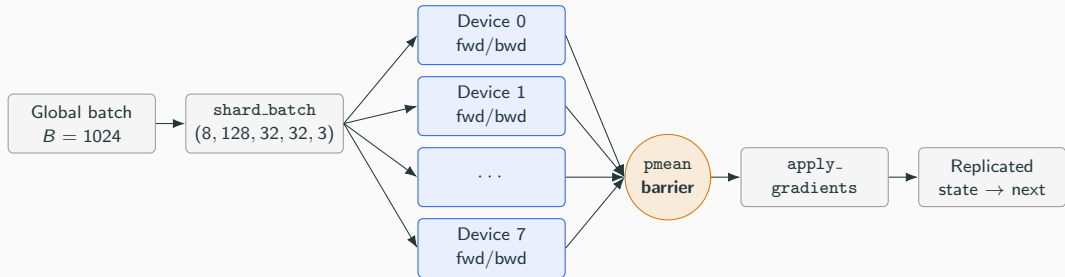
Headline training run:

- 100 epochs in 421 s
- 55.04 % top-1, 80.18 % top-5
- $\approx 15\,000$ images/sec sustained

Not a SOTA attempt — the goal is to *measure* distributed behaviour.

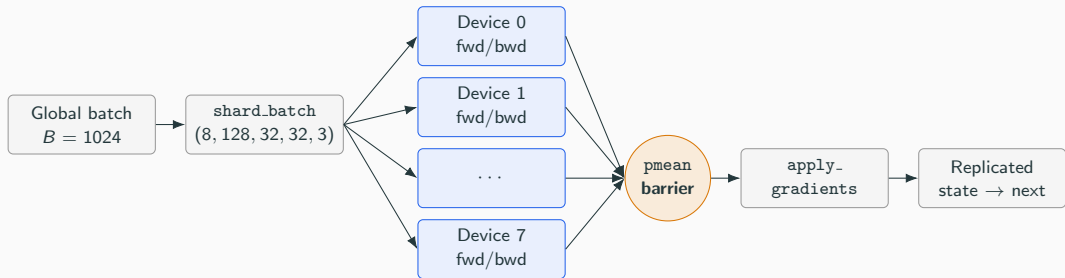


How one training step becomes distributed



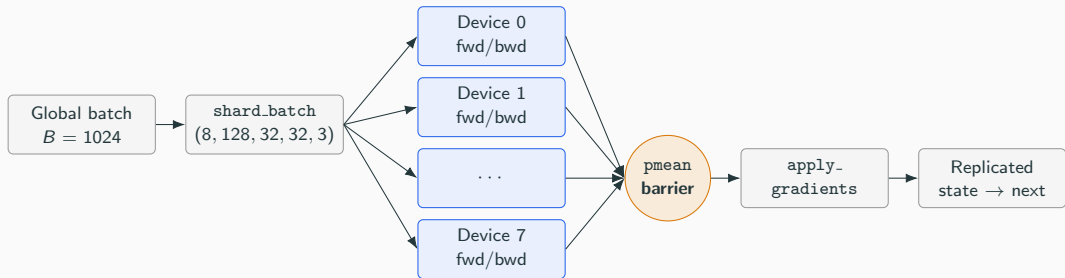
- **Replicate** params, **shard** batch, **compute** local grads, **all-reduce** via `pmean`, **update** identically.

How one training step becomes distributed



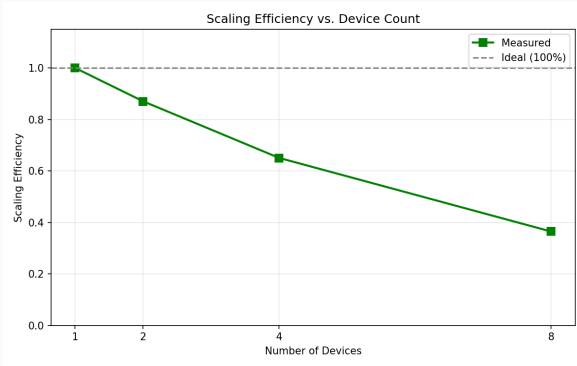
- **Replicate** params, **shard** batch, **compute** local grads, **all-reduce** via `pmean`, **update** identically.
- Mathematically equivalent to single-device training with the full global batch.

How one training step becomes distributed



- **Replicate** params, **shard** batch, **compute** local grads, **all-reduce** via `pmean`, **update** identically.
- Mathematically equivalent to single-device training with the full global batch.
- **Every device must reach `pmean` before any can proceed.** That barrier is what both experiments probe.

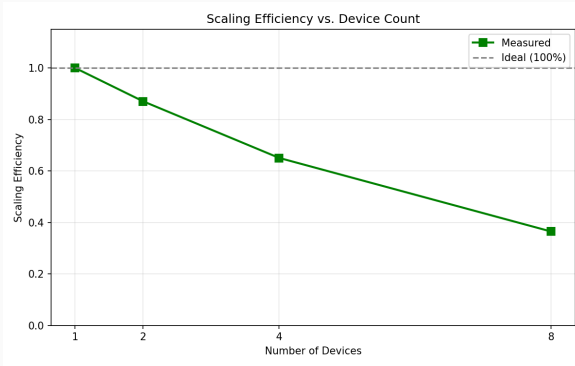
Does 8× chips really mean 8× faster training?



Setup. Fixed global batch (1024). Sweep $k = 1, 2, 4, 8$ chips. Plot the fraction of *perfect* scaling achieved ($\eta = 1.0 = \text{ideal}$).

- **k=2:** $\eta = 0.87$ — near-linear.

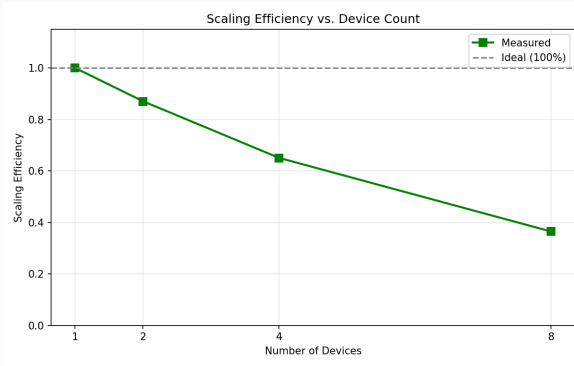
Does $8\times$ chips really mean $8\times$ faster training?



Setup. Fixed global batch (1024). Sweep $k = 1, 2, 4, 8$ chips. Plot the fraction of *perfect* scaling achieved ($\eta = 1.0 = \text{ideal}$).

- $k=2$: $\eta = 0.87$ — near-linear.
- $k=8$: η crashes to **0.36**. $8\times$ hardware, only **$2.9\times$** speedup.

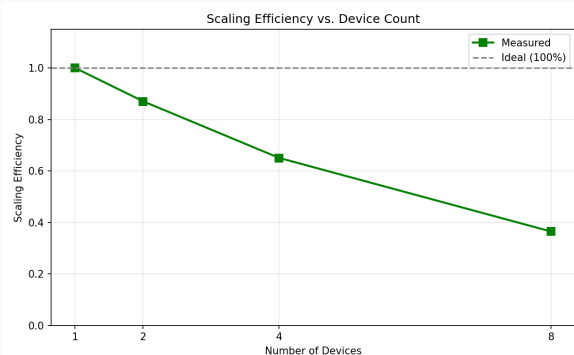
Does $8\times$ chips really mean $8\times$ faster training?



Setup. Fixed global batch (1024). Sweep $k = 1, 2, 4, 8$ chips. Plot the fraction of *perfect* scaling achieved ($\eta = 1.0 = \text{ideal}$).

- **k=2:** $\eta = 0.87$ — near-linear.
- **k=8:** η crashes to **0.36**. $8\times$ hardware, only **$2.9\times$** speedup.
- **Cause:** per-chip batch shrinks $1024 \rightarrow 128$ — MXU starves.

Does 8× chips really mean 8× faster training?

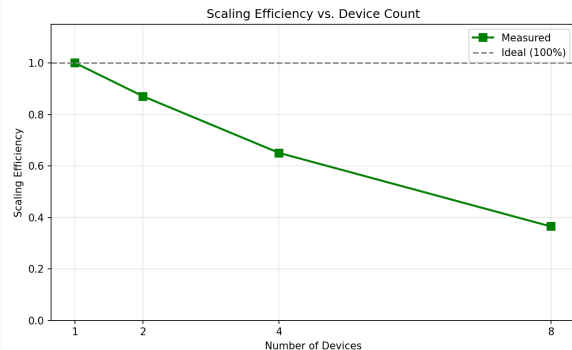


Setup. Fixed global batch (1024). Sweep $k = 1, 2, 4, 8$ chips. Plot the fraction of *perfect* scaling achieved ($\eta = 1.0 = \text{ideal}$).

- **k=2:** $\eta = 0.87$ — near-linear.
- **k=8:** η crashes to **0.36**. 8× hardware, only **2.9×** speedup.
- **Cause:** per-chip batch shrinks $1024 \rightarrow 128$ — MXU starves.
- **Model:** $T_k = \frac{T_1}{k} + R \log_2 k$, $R = 11.5$ ms/doubling

Predicted $\eta_{2/4/8} = 0.87/0.63/0.36$;
measured $0.87/0.65/0.36$ ✓

Does $8\times$ chips really mean $8\times$ faster training?

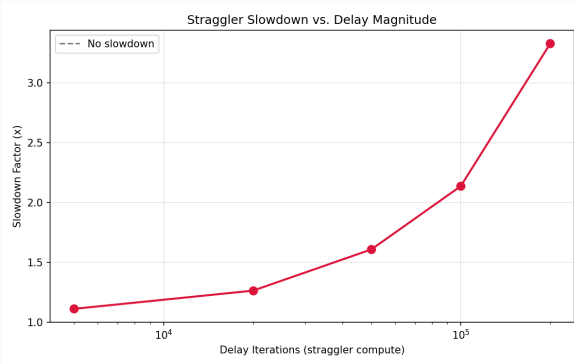


Setup. Fixed global batch (1024). Sweep $k = 1, 2, 4, 8$ chips. Plot the fraction of *perfect* scaling achieved ($\eta = 1.0 = \text{ideal}$).

- **k=2:** $\eta = 0.87$ — near-linear.
- **k=8:** η crashes to **0.36**. $8\times$ hardware, only **$2.9\times$** speedup.
- **Cause:** per-chip batch shrinks $1024 \rightarrow 128$ — MXU starves.
- **Model:** $T_k = \frac{T_1}{k} + R \log_2 k$, $R = 11.5$ ms/doubling
Predicted $\eta_{2/4/8} = 0.87/0.63/0.36$;
measured $0.87/0.65/0.36$ ✓

Takeaway: strong scaling (fixed global batch) is a batch-size trap.

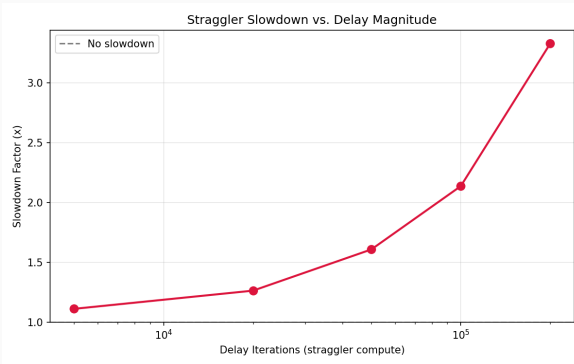
What does one slow chip cost the whole cluster?



Setup. Inject extra matmul work on *device 0 only*. Measure step time of the whole 8-chip cluster. Baseline step = 52 ms.

- Small injection (5k iters): cluster 1.1× slower.

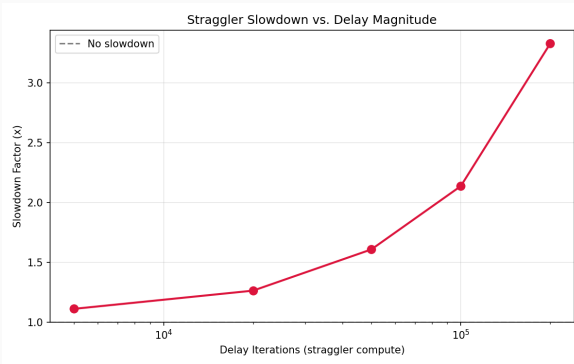
What does one slow chip cost the whole cluster?



Setup. Inject extra matmul work on *device 0 only*. Measure step time of the whole 8-chip cluster. Baseline step = 52 ms.

- Small injection (5k iters): cluster **1.1×** slower.
- Big injection (200k iters): cluster **3.33×** slower. Throughput 19.8k → 5.9k img/s.

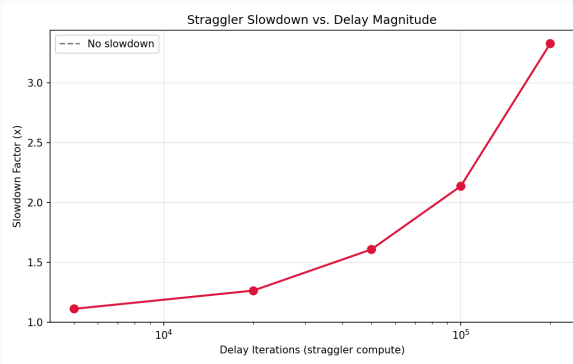
What does one slow chip cost the whole cluster?



Setup. Inject extra matmul work on *device 0 only*. Measure step time of the whole 8-chip cluster. Baseline step = 52 ms.

- Small injection (5k iters): cluster **1.1×** slower.
- Big injection (200k iters): cluster **3.33×** slower. Throughput 19.8k $\rightarrow$ 5.9k img/s.
- **Cause:** pmean is a sync barrier — 7 chips sit *idle* waiting for chip 0.

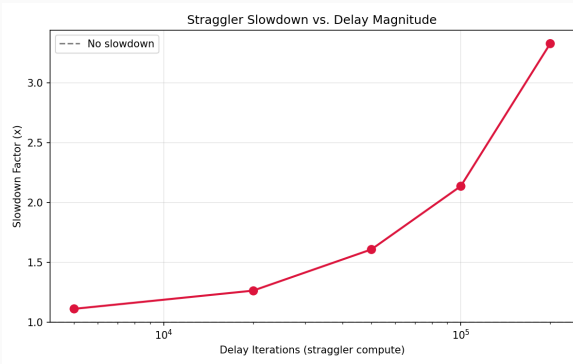
What does one slow chip cost the whole cluster?



Setup. Inject extra matmul work on *device 0 only*. Measure step time of the whole 8-chip cluster. Baseline step = 52 ms.

- Small injection (5k iters): cluster **1.1×** slower.
- Big injection (200k iters): cluster **3.33×** slower. Throughput 19.8k → 5.9k img/s.
- **Cause:** pmean is a sync barrier — 7 chips sit *idle* waiting for chip 0.
- **Model:** $\sigma(\delta) = 1 + \frac{\tau\delta}{T_{\text{base}}}$, $\tau \approx 0.6 \mu\text{s/iter}$
Predicted $\sigma(100\text{k}) = 2.17$; measured 2.14 ✓

What does one slow chip cost the whole cluster?



Setup. Inject extra matmul work on *device 0 only*. Measure step time of the whole 8-chip cluster. Baseline step = 52 ms.

- Small injection (5k iters): cluster **1.1×** slower.
- Big injection (200k iters): cluster **3.33×** slower. Throughput 19.8k → 5.9k img/s.
- **Cause:** pmean is a sync barrier — 7 chips sit *idle* waiting for chip 0.
- **Model:** $\sigma(\delta) = 1 + \frac{\tau\delta}{T_{\text{base}}}$, $\tau \approx 0.6 \mu\text{s/iter}$
Predicted $\sigma(100\text{k}) = 2.17$; measured 2.14 ✓

Takeaway: a cluster is only as fast as its slowest chip.

Takeaways

1. **Strong scaling is a small-batch trap.** Going $1 \rightarrow 8$ chips loses $2.78\times$ of the ideal speedup as per-device batch collapses and the MXU starves.

Takeaways

1. **Strong scaling is a small-batch trap.** Going $1 \rightarrow 8$ chips loses $2.78\times$ of the ideal speedup as per-device batch collapses and the MXU starves.
2. **The barrier really is the barrier.** One slow chip slows the cluster $3.33\times$ — seven of eight devices sit idle at p_{mean} . The argument for async-SGD, backup workers, elastic training.

Takeaways

1. **Strong scaling is a small-batch trap.** Going $1 \rightarrow 8$ chips loses $2.78\times$ of the ideal speedup as per-device batch collapses and the MXU starves.
2. **The barrier really is the barrier.** One slow chip slows the cluster $3.33\times$ — seven of eight devices sit idle at p_{mean} . The argument for async-SGD, backup workers, elastic training.

Thank you — questions?

Repo: github.com/Michael-Ov0/vit-jax-distributed-scaling